

Run Distribution 與 minrun Preprocessing 對 Adaptive Merge Policy 的影響

Timsort-style 與 Powersort-style Merge Policies 的實證分析

姓名：廖文瑜

2026 年 7 月

摘要

Adaptive merge sorting 會先偵測 input 中既有的 **natural runs**，再由 **merge policy** 決定後續的 **merge order**。Timsort-style policy 主要依賴 local stack rules；Powersort-style policy 則利用 **node power**，嘗試建立接近 optimal alphabetic merge tree 的結構。然而，在完整的 adaptive merge sorting pipeline 中，merge policy 通常不會直接作用於原始 natural runs，而是作用於經過 **minrun extension** 後的 **adjusted run distribution**。因此，minrun preprocessing 所造成的結構轉換，可能放大、削弱，甚至反轉兩種 merge policies 之間的 merge tree cost 差異。

本研究採用 2×2 實驗設計，比較 Timsort-style 與 Powersort-style policies 在 with-minrun 與 without-minrun 條件下的行為，並以人工設計的 run distributions 控制 input structure。資料類型包含 random、nearly_sorted、balanced_runs、skewed_runs、exponential_runs、many_tiny_runs、duplicate_heavy、alternating run patterns、one-huge-tail patterns 與 near-minrun-boundary cases。主要指標包含 **merge tree cost**、**normalized cost reduction**、**policy-gap retention ratio**、**run count**、**run entropy**、**run CV** 與 **max-run ratio**。

實驗結果顯示，Powersort-style policy 的 cost reduction 並非在所有 run distributions 中均勻出現。在 $n = 10000$ 時，nearly_sorted 的 with-minrun normalized cost reduction 為 4.70%，alternating_minrun_large_runs 為 2.33%；相對地，balanced_runs、exponential_runs 與 one_huge_tail_minrun_aware 沒有產生 cost difference，而 skewed_runs 中 Powersort-style cost 反而高出 6.97%。對 random、duplicate_heavy 與 many_tiny_runs 等高度零碎的 inputs，without-minrun condition 可觀察到 2.14% 至 5.98% 的 cost reduction，但 with-minrun 條件皆縮小至約 0.10%，顯示 minrun preprocessing 會大幅縮小兩種 merge policies 之間可觀察到的成本差異。

相關分析顯示，adjusted normalized run count 與 Powersort-style cost reduction 呈中度正相關，而 adjusted max-run ratio 呈中度負相關；但所有 correlation coefficients 的絕對值皆低於 0.4，因此只能視為探索性證據。綜合而言，adaptive merge sorting 應被視為 **run detection**、**minrun preprocessing** 與 **merge policy** 共同作用的系統。若只比較 merge policy 的理論性質，而忽略 adjusted runs 的實際結構，可能高估或低估 policy 的差異。

Keywords : adaptive merge sorting 、 natural runs 、 Timsort 、 Powersort 、 minrun 、 minrun preprocessing 、 merge policy 、 merge tree cost 、 run entropy

目錄

1. Introduction
 2. Background and Related Work
 3. Methodology
 4. Results
 5. Discussion
 6. Threats to Validity and Limitations
 7. Conclusion
- References
Appendix

1. Introduction

1.1 Research Background

傳統的 merge sort 通常依固定方式切分 input，再逐層執行 merge，不會主動利用資料中已存在的局部順序。Natural mergesort 則先偵測 maximal monotonic segments，並將它們視為 initial runs。當 input 已部分排序時，run count 可能遠小於 element count，使演算法有機會降低後續 merge workload。

Auger 等人以 run count ρ 分析 Timsort，證明其 Python 版本的執行時間為 $O(n + n \log \rho)$ 。此結果顯示，run count 是解釋 Timsort 對 partially sorted inputs 之 adaptivity 的自然參數 [2]。Buss 與 Knop 亦以 stable natural merge sorting 為研究框架，將 merge strategy 與 merge cost 作為分析核心 [3]。

真實資料未必完全 random，而可能具有下列 run structures：

- long ascending or descending regions；
- partially sorted blocks；
- many tiny or fragmented runs；
- duplicate-heavy nondecreasing regions；
- adjacent runs with highly unequal lengths；
- one dominant run combined with several smaller runs。

因此，adaptive merge sorting 的 cost 不只與 input size n 有關，也受到 run-length sequence、run positions 與 merge order 影響。

1.2 Problem Statement

Timsort-style policy 將 natural runs 依序 push 到 stack，並使用 stack 頂端的長度條件決定 merge 時機。Powersort-style policy 則根據 node power 管理 stack，使 merge tree 接近 nearly-optimal alphabetic merge tree [1]。從理論保證出發，容易產生一個直覺：當 run lengths 不規則時，Powersort-style policy 應更容易形成較低 cost 的 merge tree。

然而，實際 processing pipeline 是：

Input

- Natural Run Detection
- Descending-Run Reversal
- Optional minrun Extension
- Adjusted Run Distribution
- Merge Policy
- Merge Tree
- Sorted Output

Merge policy 真正接收到的是 **adjusted run distribution**，而不是未經處理的 natural run distribution。對 random 或 many_tiny_runs 等 inputs，minrun extension 可能將數千個 tiny runs 轉換為約 n/minrun 個接近等長的 adjusted runs。此時，原始 input 即使高度零碎，兩種 policy 實際面對的 run structure 也可能相當 regular。

因此，本研究的重點不是再次證明 Powersort 的理論界線，而是分析：

哪些 run-distribution features 會放大或削弱 Timsort-style 與 Powersort-style policies 的 visible cost difference，以及 minrun preprocessing 如何改變這項關係？

1.3 Research Questions

本研究提出以下四個 research questions：

RQ1：不同 run-distribution features 如何影響 Timsort-style 與 Powersort-style policies 的 merge tree cost difference？

RQ2：Policy gap 與 natural run features 或 adjusted run features，何者呈現較明顯的關聯？

RQ3：minrun preprocessing 是否會壓縮 without-minrun 條件中原本可觀察到的 policy gap？

RQ4：Highly unequal run lengths 是否必然使 Powersort-style policy 具有正向 cost reduction？

1.4 Contributions

本研究的主要貢獻如下：

1. 使用 2×2 實驗設計分離 merge policy 與 minrun preprocessing 的作用。
2. 同時量測 natural runs 與 adjusted runs，避免只以原始 input structure 解釋 policy behavior。
3. 使用 normalized cost reduction 與 policy-gap retention ratio 描述 minrun 前後的差異。
4. 建立 raw、minrun-aware 與 threshold stress cases，區分 fragmentation、heterogeneity 與 dominance。
5. 提供 merge tree case study，說明相同 adjusted runs 如何因 policy 不同而形成不同 weighted path length。

2. Background and Related Work

2.1 Natural Runs and Adaptive Merge Sorting

Natural run 是 input 中連續的 maximal monotonic segment。例如：

1 2 5 8 | 3 4 7 | 2 6 9

可被辨識為三個 ascending runs，lengths 分別為 4、3 與 3。對 descending runs，sorting implementation 通常會先 reverse，使後續 merge routine 一律處理 nondecreasing runs。

Auger 等人將 Timsort 的高階機制描述為：先以 greedy 方式將序列分解成單調 runs，再依特定局部規則進行兩兩 merge。其分析進一步證明 Python Timsort 的執行時間為 $O(n + n \log \rho)$ ，其中 ρ 為 run count [2]。這提供了本研究使用 natural run count 與 adjusted run count 作為結構特徵的理論動機。

2.2 minrun Extension

Timsort 的核心機制以 natural runs 與以 stack 為基礎的 merge strategy 為基礎；實務上的 Timsort 另外使用 insertion sort 啟發式規則，避免初始 runs 過短 [2]。本研究將此步驟明確建模為 minrun extension：當偵測到的 natural run 短於 minrun 時，使用 stable binary insertion sort 將該 run 向右延伸。

本研究採用固定 minrun 計算方式。對較大的 n ，程式計算出的 minrun 落在 32 至 64 之間。這是本研究的實作設定，用來建立 with-minrun 與 without-minrun 的控制比較，而不是對任何特定正式實作的逐行重製。

minrun extension 可能直接改變 merge policy 所接收到的輸入結構，包括：

1. 降低初始 run count；
2. 將大量極短 runs 轉換為接近 minrun 的 adjusted runs；
3. 改變 run entropy、run CV 與 max-run ratio；
4. 改變兩種 merge policy 最後建立的 merge trees。

因此，本研究分別量測 natural run distribution 與 adjusted run distribution。

2.3 Timsort-Style Merge Policy

Timsort-style policy 將待處理 runs 存入 stack，並根據 stack 頂端 runs 的長度條件執行合併處理。其 merge 決策只依賴有限的局部 stack 資訊。Auger 等人分析了修正後 Python Timsort 的 stack 規則與最壞情況複雜度，並指出 Timsort 可在即時偵測 runs 的同時，使用局部性質決定 merge [2]。

Buss 與 Knop 將 Timsort 納入 stable merge strategies 的統一分析框架，並以 merge cost 比較不同 merge order 規則 [3]。本研究據此使用 Timsort-style 一詞，表示實作保留 natural run detection、固定 minrun extension 與主要 stack collapse 規則，但不等同於完整的正式版 Timsort。

本研究未包含：

- galloping mode；
- production-grade temporary buffer management；
- cache-aware optimization；
- 特定標準函式庫中的所有 heuristics。

2.4 Powersort-Style Merge Policy

Munro 與 Wild 提出的 Powersort 會計算相鄰 runs 之間的 **node power**。當新的 run 被 push 到 stack 時，較高 power 的 boundaries 會先觸發 merge，使 stack 中的 powers 維持特定順序。此方法能在不明確儲存完整 merge tree 的情況下，線上建立 nearly-optimal alphabetic merge tree [1]。

對 run lengths $L_1, L_2, \dots, L_r$ 與 total length $n = \sum_i L_i$ ，定義 run entropy：

$$H(\mathbf{L}) = \sum_{i=1}^r \frac{L_i}{n} \log_2 \left(\frac{n}{L_i} \right).$$

Munro 與 Wild 證明 Powersort 對 runs $L_1, \dots, L_r$ 的 comparison count 可控制在 $nH(\mathbf{L}) + O(n)$ ，並給出 $nH(\mathbf{L}) + 3n$ 的具體 upper bound [1]。此結果代表 Powersort 的 merge order 在最高次項上能適應既有 runs，但它並不表示 Powersort 對每一個有限的 run length 序列都必然嚴格優於其他啟發式規則 (heuristic)。

2.5 Merge Tree Cost

本研究的主要 response variable 為 **merge tree cost**：

$$C(T) = \sum_{v \in \text{internal nodes}} \text{size}(v) = \sum_{i=1}^r L_i d_i,$$

其中 d_i 為 initial run i 在 merge tree 中的 depth。每當一個 run 參與一次 merge，其 elements 會被計入一次，因此 $C(T)$ 等價於 initial runs 的 weighted path length。

Buss 與 Knop 將每次 merge A 與 B 的 cost 定義為 $|A| + |B|$ ，再將所有 merges 的 cost 加總。他們指出，此 merge cost upper-bounds comparison count，並可作為 runtime 的重要模型 [3]。Munro 與 Wild 的 Powersort 分析也以 run lengths 與 merge tree quality 為核心 [1]。

Merge tree cost 不等於實際執行時間，但能將研究焦點集中於 merge policy 所造成的 merge tree 結構差異。因此，本研究以 $C(T)$ 作為主要指標，並將 merge comparisons 留作輔助指標。

3. Methodology

3.1 Scope of the Experiment

本研究比較 **Timsort-style** 與 **Powersort-style** merge policies，而非兩套完整的正式排序系統。所有條件共用相同的：

- natural run detection；
- descending-run reversal；
- binary insertion sort for minrun extension；
- stable two-way merge implementation；
- synthetic input generators。

兩種 policies 的主要差異只存在於 merge-order decision。此設計可降低其他 implementation details 對 research questions 的干擾。

3.2 2×2 Experimental Design

Preprocessing condition	Timsort-style policy	Powersort-style policy
With minrun	timsort_style_with_minrun	powersort_style_with_minrun
Without minrun	timsort_style_without_minrun	powersort_style_without_minrun

With-minrun 條件模擬包含 minrun extension 的 adaptive sorting pipeline；without-minrun 條件則作為 ablation，使 merge policy 直接作用於 run detection 與 descending-run reversal 後的 natural runs。

3.3 Input Sizes and Repetitions

實驗採用三種 input sizes：

$$n \in \{1000, 5000, 10000\}.$$

每一種 data type 在每個 size 下執行 5 次 trial。random 與 nearly_sorted 使用固定的 random seed 42，以確保可重現性。總實驗數為：

$$3 \text{ sizes} \times 12 \text{ data types} \times 5 \text{ trials} \times 4 \text{ conditions} = 720.$$

每一次輸出皆與 Python 內建 `sorted` 的結果比較；720 組 experiments 全部通過正確性驗證。

3.4 Synthetic Run Distributions

Data type	Structural purpose
random	產生大量短 natural runs，測試 strong minrun preprocessing
nearly_sorted	以少量隨機交換建立部分排序結構
balanced_runs	八個等長 runs，作為規則基準案例
skewed_runs	一個大型 run 搭配數個小型 runs
exponential_runs	run lengths 呈幾何成長
many_tiny_runs	大量長度為 2 的零碎 runs
duplicate_heavy	以少量 unique values 形成大量重複值的結構
alternating_small_large_raw_runs	小型與大型 runs 交替，用來觀察 minrun extension 是否使 run lengths 趨於一致
alternating_minrun_large_runs	minrun 長度與大型 runs 交替出現
one_huge_tail_raw_runs	多個小型原始 runs 後接一個巨大尾端 run
one_huge_tail_minrun_aware	minrun 長度的前綴 runs 後接一個巨大尾端 run
near_minrun_boundary	run lengths 在 minrun 門檻上下交替

Raw stress cases 用來觀察 minrun 是否改寫原始 run distribution；minrun-aware cases 則刻意避免 run 被 extension，以隔離 merge policy behavior。

3.5 Run-Distribution Features

本研究擷取下列 features：

1. **Run count**：initial 或 adjusted runs 的數量。
2. **Normalized run count**： r/n 。
3. **Run entropy bound**： $nH(L)$ 。
4. **Run coefficient of variation (run CV)**：

$$CV = \frac{\sigma_L}{\mu_L}.$$

5. **Max-run ratio**：

$$M = \frac{\max_i L_i}{n}.$$

6. **Max-to-average ratio**：最大 run length 除以平均 run length。
7. **Insertion ratio**：被 binary insertion sort 涵蓋的 elements 占 n 的比例。

8. **Run-count retention ratio :**

$$R_{\text{run}} = \frac{r_{\text{adjusted}}}{r_{\text{natural}}}.$$

9. **Entropy change ratio :**

$$R_H = \frac{nH_{\text{adjusted}}}{nH_{\text{natural}}}.$$

$R_H < 1$ 表示經過 minrun extension 後，adjusted runs 的 run entropy 低於 natural runs； $R_H = 1$ 表示前後相同； $R_H > 1$ 則表示 run entropy 增加。Entropy 的變化同時受到 run count 與 run-length distribution 影響，因此 $R_H > 1$ 不應單獨解讀為 runs 必然更接近等長，而應搭配 run count、run CV 與 max-run ratio 共同判斷。

3.6 Response Variables

3.6.1 Normalized Powersort-Style Cost Reduction

定義：

$$A = \frac{C_T - C_P}{C_T},$$

其中 C_T 與 C_P 分別為 Timsort-style 與 Powersort-style 的 merge tree cost。

- $A > 0$: Powersort-style cost 較低；
- $A = 0$: 兩者 cost 相同；
- $A < 0$: Powersort-style cost 較高。

本報告使用 **cost reduction** 而非直接使用 **advantage**，以避免預設 Powersort-style 的成本必然較低。

3.6.2 Policy-Gap Retention Ratio

令：

$$\Delta_{\text{with}} = C_{T,\text{with}} - C_{P,\text{with}},$$

$$\Delta_{\text{without}} = C_{T,\text{without}} - C_{P,\text{without}}.$$

定義：

$$R_{\Delta} = \frac{\Delta_{\text{with}}}{\Delta_{\text{without}}}.$$

若 without-minrun 條件下的 policy gap 為 0，則 R_{Δ} 無法定義，因為分母為 0。若 $R_{\Delta} < 0$ ，表示加入 minrun preprocessing 後，兩種 merge policies 的相對成本高低發生反轉；例如原本 Powersort-style 成本較低，加入 minrun 後則變成 Timsort-style 成本較低，反之亦然。因此，policy-gap retention ratio 不能單獨解讀，必須同時檢視 with-minrun 與 without-minrun 的原始 policy gaps。

3.7 Correlation Analysis

本研究使用 Pearson 相關係數，初步觀察各項 run distribution 特徵與 Powersort-style Cost Reduction 之間是否存在線性關聯。例如，本研究檢查 adjusted run count 增加時，成本降低率是否也有上升傾向。

分析資料由 12 種資料類型與 3 種輸入規模組成，共有 $12 \times 3 = 36$ 個彙整觀察值。每個觀察值代表特定資料類型在特定輸入規模下的平均實驗結果。由於部分實驗在 without-minrun condition 下的 policy gap 為 0，無法計算 policy-gap retention ratio，因此該指標只有 25 個有效觀察值。

Pearson 相關係數只能描述兩個變數是否呈現線性共同變化，不能證明其中一個變數造成另一個變數改變。此外，相同資料產生器在不同輸入規模下的結果可能具有相似結構，並非完全獨立；本研究使用的資料類型也是人工設計，而非從真實資料母體中隨機抽樣。因此，相關分析僅用於尋找可能的結構模式，不作為因果關係、統計顯著性或特徵重要性的正式證明。

4. Results

4.1 Powersort-Style Cost Reduction With and Without minrun

Figure 1 比較 $n = 10000$ 時，兩種 policies 在 with-minrun 與 without-minrun conditions 下的 normalized merge-cost reduction。數值為 5 次 trials 的平均。

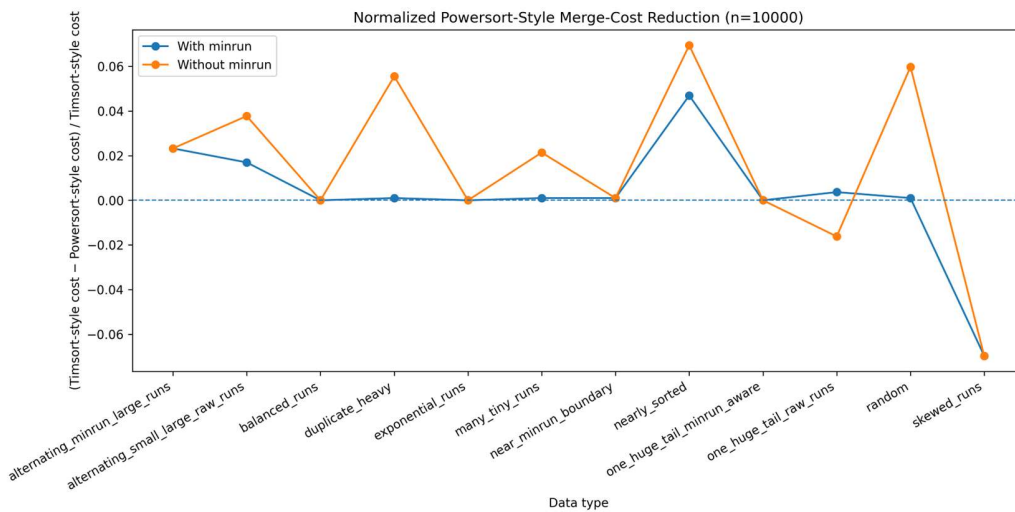


Figure 1. Normalized Powersort-style merge-cost reduction with and without minrun preprocessing at $n = 10000$.

結果可分為四類。

4.1.1 Stable Positive Gap

nearly_sorted 在三種 n 下均呈現 positive cost reduction：with-minrun 為 3.82% 至 5.57%，without-minrun 為 4.21% 至 6.95%。在 $n = 10000$ 時，with-minrun 為 4.70%，是 main experiment 中最明顯的 positive result。

`alternating_minrun_large_runs` 在 $n = 5000$ 與 $n = 10000$ 分別為 2.72% 與 2.33%，且 `with-minrun` 與 `without-minrun` 完全相同。原因是所有 `natural runs` 都已不短於 `minrun`，因此 `minrun extension` 沒有改變 `adjusted run distribution`。

4.1.2 Gap Suppressed by minrun

在 $n = 10000$ ：

- `random`：without-minrun 為 5.98%，with-minrun 降至 0.10%；
- `duplicate_heavy`：5.56% 降至 0.10%；
- `many_tiny_runs`：2.14% 降至 0.10%；
- `alternating_small_large_raw_runs`：3.78% 降至 1.70%。

這些結果支持 RQ3：當 `minrun preprocessing` 將大量 `short runs` 轉換為較少且接近等長的 `adjusted runs` 時，merge policies 所形成的 merge tree cost 差異會明顯縮小。

4.1.3 No Observable Gap

`balanced_runs`、`exponential_runs` 與 `one_huge_tail_minrun_aware` 在 $n = 10000$ 的 `with-minrun` 與 `without-minrun` 條件下皆為 0。此結果顯示，等長、幾何成長或主導尾端等單一結構標籤，皆不足以保證兩種 policies 形成不同 `weighted path length`。

4.1.4 Negative Gap and Ranking Reversal

`skewed_runs` 在 $n = 5000$ 與 $n = 10000$ 的 Powersort-style cost 分別高出 4.54% 與 6.97%，且 `minrun` 沒有改變其 run structure。這直接否定「只要存在一個 `dominant run`，Powersort-style 就一定較好」的簡單直覺。

`one_huge_tail_raw_runs` 更出現正負號反轉：在 $n = 10000$ ，without-minrun 為 -1.62%，with-minrun 則為 +0.37%。這表示 `minrun preprocessing` 不只可能壓縮 policy gap，也可能改變兩種 policies 的 relative ranking。

4.1.5 Numerical Summary at $n = 10000$

Data type	With minrun	Without minrun	Gap retention
<code>alternating_minrun_large_runs</code>	2.33%	2.33%	100.00%
<code>alternating_small_large_raw_runs</code>	1.70%	3.78%	41.67%
<code>balanced_runs</code>	0.00%	0.00%	undefined
<code>duplicate_heavy</code>	0.10%	5.56%	1.13%
<code>exponential_runs</code>	0.00%	0.00%	undefined
<code>many_tiny_runs</code>	0.10%	2.14%	2.96%
<code>near_minrun_boundary</code>	0.10%	0.10%	100.00%
<code>nearly_sorted</code>	4.70%	6.95%	62.02%

Data type	With minrun	Without minrun	Gap retention
one_huge_tail_minrun_aware	0.00%	0.00%	undefined
one_huge_tail_raw_runs	0.37%	-1.62%	-16.46%
random	0.10%	5.98%	1.04%
skewed_runs	-6.97%	-6.97%	100.00%

4.2 Retention of the Policy Gap After minrun

Figure 2 顯示 R_d 。只有 without-minrun gap 非零的 data types 才能定義此 ratio。

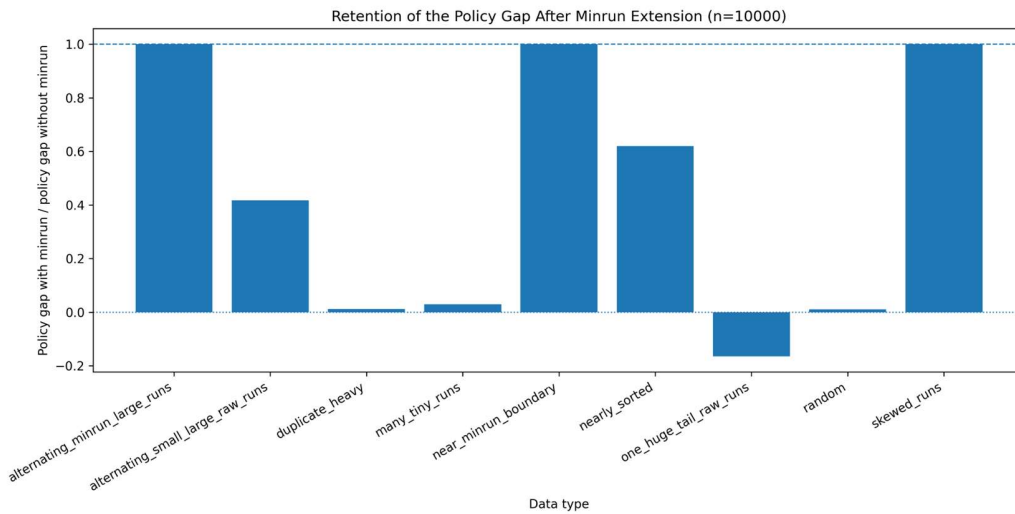


Figure 2. Retention of the policy gap after minrun extension at $n = 10000$.

Highly fragmented inputs 的 retention 非常低：

- random : 1.04% ;
- duplicate_heavy : 1.13% ;
- many_tiny_runs : 2.96% 。

在這三個 cases 中，natural run counts 分別約為 4143、4057 與 5000；經過 minrun extension 後皆變成 250 個 adjusted runs。也就是說，minrun preprocessing 在 merge policy 開始作用前，已消除超過 93% 的 original run boundaries。

nearly_sorted 保留 62.02% 的 original gap，alternating_small_large_raw_runs 保留 41.67%。這兩者經過 minrun extension 後仍保留一定程度的 run-length heterogeneity，因此 policy gap 雖縮小，卻沒有消失。

alternating_minrun_large_runs、near_minrun_boundary 與 skewed_runs 的 retention 接近 1，但意義不同：前兩者是 gap 被完整保留；skewed_runs 則是 negative gap 被完整保留。因此，high retention 不代表 Powersort-style 表現較好，只代表 minrun 沒有改變 relative cost difference。

4.3 Associations Between Run Features and Cost Reduction

Figure 3 顯示 with-minrun cost reduction 與 run-distribution features 的 Pearson correlations。

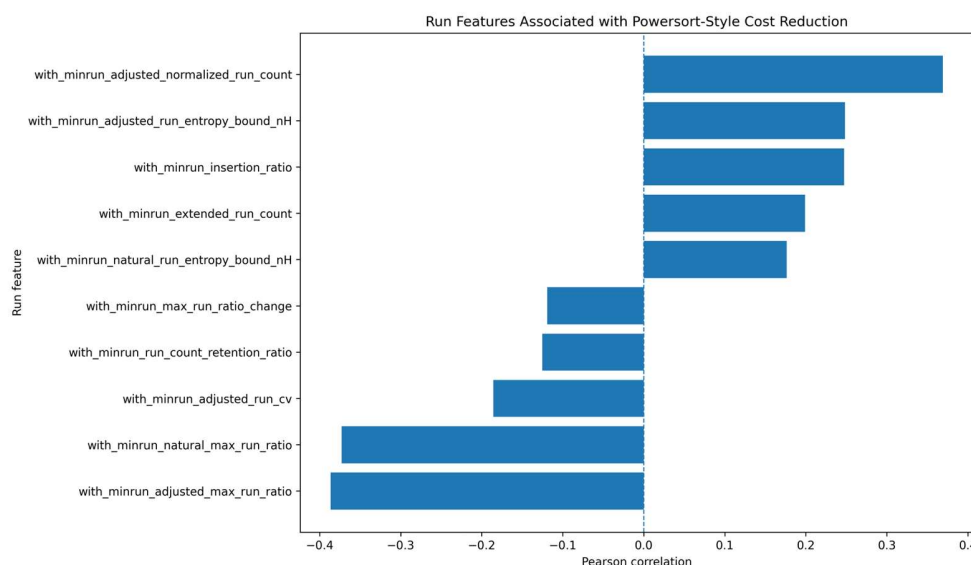


Figure 3. Pearson correlations between run-distribution features and with-minrun Powersort-style cost reduction.

Absolute correlation 最高的 features 仍低於 0.4：

- adjusted max-run ratio： $r = -0.386$ ；
- natural max-run ratio： $r = -0.373$ ；
- adjusted normalized run count： $r = 0.369$ ；
- adjusted nH ： $r = 0.248$ ；
- insertion ratio： $r = 0.247$ 。

這些結果支持兩個較保守的解釋。

第一，較多 **adjusted runs** 可能增加兩種 **policies** 建立不同 **merge trees** 的機會。但相關係數只有 0.369，表示 adjusted run count 並非充分條件。random 與 many_tiny_runs 雖有 250 個 adjusted runs，因 lengths 幾乎完全相同，with-minrun gap 仍只有 0.10%。

第二，**max-run ratio** 越高，**Powersort-style cost reduction** 傾向越低。此關連與 skewed_runs、one_huge_tail_minrun_aware 的結果一致。一個 dominant run 可能限制 alphabetic merge tree 的 feasible structures，或使簡化的 Timsort-style policy 在特定序列上形成較低 cost。

此外，policy-gap retention ratio 與 run-count retention 的 correlation 為 0.834，與 entropy retention 的相關係數為 0.722，與 insertion ratio 的相關係數為 -0.463。雖然有效 observations 只有 25 個，這組結果仍與研究假設一致：minrun preprocessing 保留越多 original run structure，policy gap 通常也保留得越多。

4.4 Case Study: Different Merge Trees on the Same Adjusted Runs

為直接展示 merge policy 對 merge tree 的影響，本節使用 run-length sequence：

$$[64, 256, 256, 64, 64], \quad n = 704.$$

此時 fixed minrun 為 44，所有 runs 皆不需要 extension，因此兩種 policies 接收到完全相同的 adjusted run distribution。

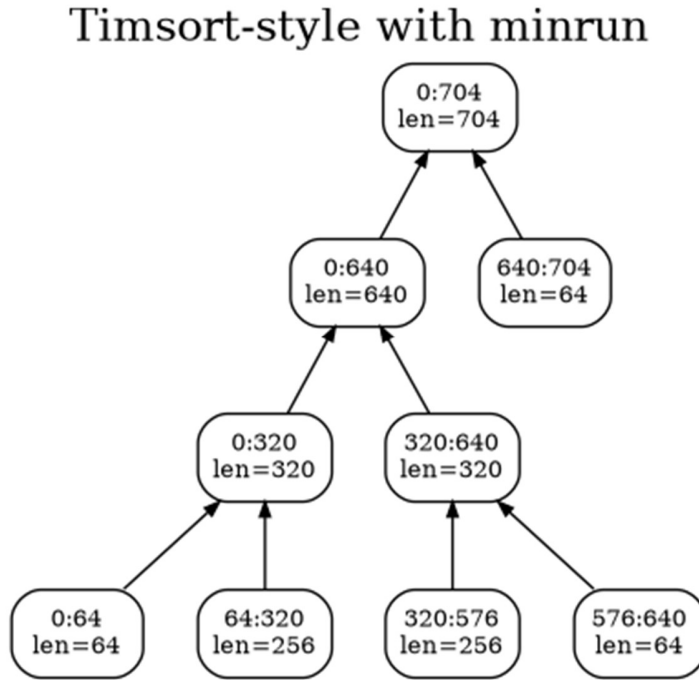


Figure 4(a). Timsort-style merge tree for the illustrative run sequence.

Powersort-style with minrun

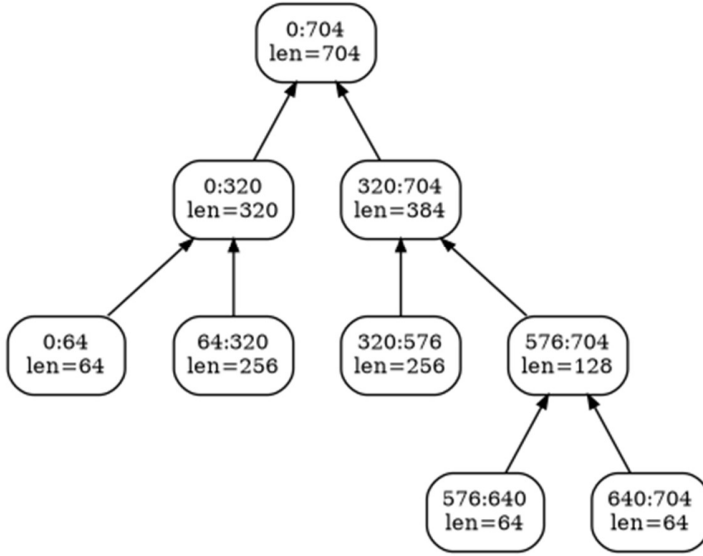


Figure 4(b). Powersort-style merge tree for the same run sequence.

Timsort-style policy 的 merge tree cost 為：

$$C_T = 1984,$$

Powersort-style policy 的 merge tree cost 為：

$$C_P = 1536.$$

因此：

$$A = \frac{1984 - 1536}{1984} = 22.58\%.$$

Powersort-style tree 將較大的 256-length runs 放在較淺的 depth，並讓三個 64-length runs 承擔較深的 paths，因此降低 weighted path length。此案例說明，merge policy difference 不只是 stack operations 的差異，也會直接決定每個 initial run 的 elements 被重複處理多少次。

不過，此序列是為說明機制而選擇的案例，不代表主要實驗的平均改善幅度。主實驗的最高 with-minrun average cost reduction 約為 5.57%，低於此 illustrative case 的 22.58%。

5. Discussion

5.1 Powersort-Style Improvement Is Conditional

本研究最重要的結果，是 Powersort-style policy 並未在所有 synthetic run distributions 中產生 positive cost reduction。即使 Powersort 具有 nearly-optimal theoretical guarantee，簡化的 online implementation 在某些 finite run sequences 上仍可能與 Timsort-style 形成相同 cost，甚至形成較高 cost。

這不否定 Powersort 的 theoretical result。Munro 與 Wild 的 guarantee 是相對於 entropy-based bound 與 additive linear terms，而不是宣稱 Powersort 對每一個 finite input 都嚴格優於任何 other heuristic [1]。實驗中的 negative gap 顯示，worst-case guarantee、average behavior 與 instance-by-instance comparison 是不同層次的 claims。

5.2 Adjusted Runs Are More Relevant Than Raw Fragmentation

random、duplicate_heavy 與 many_tiny_runs 在 natural-run level 高度 fragmented，但經過 minrun extension 後都變成 250 個 equal-length adjusted runs。在此 condition 下，with-minrun cost reduction 幾乎消失。

因此，單純描述 input 「很亂」或 natural run count 很高，並不足以解釋 merge policy behavior。真正直接進入 policy decision 的，是 adjusted run count、adjusted run-length ratios 與 run positions。

5.3 Fragmentation and Dominance Are Different Structures

本研究原先的直覺是「run distribution 越 irregular，Powersort-style 越能展現優勢」，但結果顯示至少需要區分兩類 irregularity：

1. **fragmented heterogeneity**：many runs with different lengths；
2. **dominant-run structure**：one run occupies most of the input。

Fragmented heterogeneity 可能提供較多 merge-order alternatives；dominant-run structure 則可能強烈限制 alphabetic tree 的 shape。skewed_runs 與 one_huge_tail_minrun_aware 顯示，高 run CV 或高 max-run ratio 並不等於較大的 positive policy gap。

5.4 Entropy Is a Bound, Not a Complete Predictor

Run entropy nH 提供與 run-length distribution 有關的 information-theoretic scale，但不包含 runs 在 input 中的 order。兩組具有相同 run-length multiset 的 inputs，若 positions 不同，online merge policy 可能建立不同 trees。另一方面，兩種 policies 也可能在 high-entropy distribution 上形成相同 merge tree cost。

因此，run entropy 適合描述整體 distribution complexity 或作為 normalization reference，卻不能單獨預測特定 policy gap。未來可加入 position-sensitive features，例如 adjacent length ratios、local peaks、prefix/suffix dominance 與 optimal alphabetic merge tree cost。

5.5 Implications for Benchmark Design

若 benchmark 只使用 random arrays，可能得到「with-minrun 下兩種 policies 幾乎沒有差異」的結論；若只使用特別設計的 unequal runs，又可能高估 policy gap。較完整的 adaptive sorting benchmark 應同時包含：

- preprocessing-heavy fragmented inputs；
- partially sorted real-like inputs；
- minrun-aware nonuniform runs；
- dominant-run cases；
- near-minrun threshold cases；
- natural runs extracted from real-world data。

此外，evaluation 應同時報告 natural run distribution 與 adjusted run distribution，否則難以區分 observed difference 究竟來自 input structure、minrun preprocessing 或 merge policy。

6. Threats to Validity and Limitations

6.1 Construct Validity

本研究以 merge tree cost 作為主要 metric。它能反映 elements 在 merge tree 中被重複處理的總量，但不等同於 wall-clock runtime。實際 performance 還受到 comparison cost、memory allocation、cache locality、branch prediction、copy routine 與 galloping 等因素影響。

6.2 Implementation Validity

本研究程式是 Timsort-style 與 Powersort-style merge policies 的 research model，不是任何 standard library implementation 的完整重製。主要 simplifications 包含：

- 未實作 galloping mode；
- 未使用 production-grade temporary buffer management；
- 未進行 cache-aware optimization；
- 使用 fixed-minrun model；
- node power 與 stack policy 只重製 core concepts，而非特定 source code version。

因此，本研究只能對此 simplified model 的 merge-tree behavior 下結論。

6.3 Internal Validity

四個 conditions 共用相同 input instance，可降低 input variation 造成的 confounding；程式也對每次 output 執行 sorted-result verification。然而，generator、run detection 與 merge policies 皆由同一份程式實作，仍可能存在 shared implementation errors。

為降低此風險，程式額外檢查：

- `adjusted runs` 是否完整 partition $[0, n)$ ；
- `merge count` 是否等於 `initial run count` 減一；
- 每個 `condition` 是否得到正確 `sorted output`；
- `merge tree cost` 是否等於所有 `merged interval sizes` 的總和。

6.4 External Validity

本研究主要使用 `synthetic run distributions`，不能直接代表 `database records`、`time-series logs`、`UI data`、`web tables` 或其他 `real-world inputs`。不同 `key distributions`、`comparison functions` 與 `duplicate patterns` 都可能改變 `comparison count` 與 `galloping behavior`。

6.5 Statistical Conclusion Validity

每個 `condition` 只有 5 次 `trials`；部分 `generators` 為 `deterministic`，因此 `repeated trials` 不會增加新的 `run structure`。Pearson correlation 只有 36 個 `aggregated observations`，且相同 `generator` 在三個 n 下並非 `independent samples`。Correlation results 應視為 `exploratory analysis`，而不是 `statistical significance` 或 `causal relationship` 的證明。

6.6 Retention Ratio Instability

當 `without-minrun gap` 接近 0 時，`policy-gap retention ratio` 會不穩定；`gap` 為 0 時則無法定義。Negative retention 也可能代表 `ranking reversal`，而不是單純的 `gap reduction`。因此，Figure 2 必須與 Figure 1 及原始 `cost values` 一起解讀。

7. Conclusion

本研究分析 `run distribution` 與 `minrun preprocessing` 如何影響 `Timsort-style` 與 `Powersort-style merge policies` 的 `visible cost difference`。實驗不支持「`Powersort-style` 對所有 `irregular inputs` 都較好」的簡單命題，而得到以下四項結論。

第一，`policy gap` 與 `adjusted run distribution` 密切相關。對 `random`、`duplicate_heavy` 與 `many_tiny_runs`，`minrun extension` 將數千個 `natural runs` 壓縮為 250 個 `equal-length adjusted runs`，使原本 2.14% 至 5.98% 的 `cost reduction` 幾乎消失至 0.10%。

第二，`structure retention` 不等於 `positive improvement retention`。`alternating_minrun_large_runs` 保留約 2.33% 的 `positive gap`，但 `skewed_runs` 也完整保留約 -6.97% 的 `negative gap`。`minrun` 是否改變 `gap`，與 `gap` 本身的 `sign` 是兩個不同問題。

第三，不同 `irregularity types` 對 `merge policy` 的影響不同。Many `heterogeneous runs` 較可能讓 `policies` 建立不同 `merge trees`；one `dominant run` 則可能限制 `tree freedom`，使兩種 `policies cost` 相同，或使 `Timsort-style` 在特定 `sequence` 上較低。

第四，單一 run feature 無法完整預測 policy gap。Adjusted normalized run count 與 cost reduction 的 correlation 為 0.369，adjusted max-run ratio 為 -0.386，皆只屬 moderate association。未來研究需要加入 position-sensitive features、optimal alphabetic merge cost 與 multivariable analysis。

綜合而言，adaptive merge sorting 不應將 minrun preprocessing 與 merge policy 視為彼此獨立的 components。Natural run detection 決定 initial structure，minrun extension 重新塑造 adjusted runs，merge policy 再於 adjusted run distribution 上建立 merge tree。只有同時分析這三個 stages，才能解釋 theoretical advantage 何時會在 experiment 中顯現、被削弱或完全消失。

References

- [1] J. I. Munro and S. Wild, “Nearly-Optimal Mergesorts: Fast, Practical Sorting Methods That Optimally Adapt to Existing Runs,” in *Proceedings of the 26th Annual European Symposium on Algorithms (ESA 2018)*, LIPIcs, vol. 112, pp. 63:1–63:16, 2018, doi: 10.4230/LIPIcs.ESA.2018.63.
- [2] N. Auger, V. Jugé, C. Nicaud, and C. Pivoteau, “On the Worst-Case Complexity of TimSort,” in *Proceedings of the 26th Annual European Symposium on Algorithms (ESA 2018)*, LIPIcs, vol. 112, pp. 4:1–4:13, 2018, doi: 10.4230/LIPIcs.ESA.2018.4.
- [3] S. Buss and A. Knop, “Strategies for Stable Merge Sorting,” in *Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2019)*, pp. 1272–1290, 2019, doi: 10.1137/1.9781611975482.78.

Appendix A. Structural Effects of minrun

A.1 Run-Entropy Retention

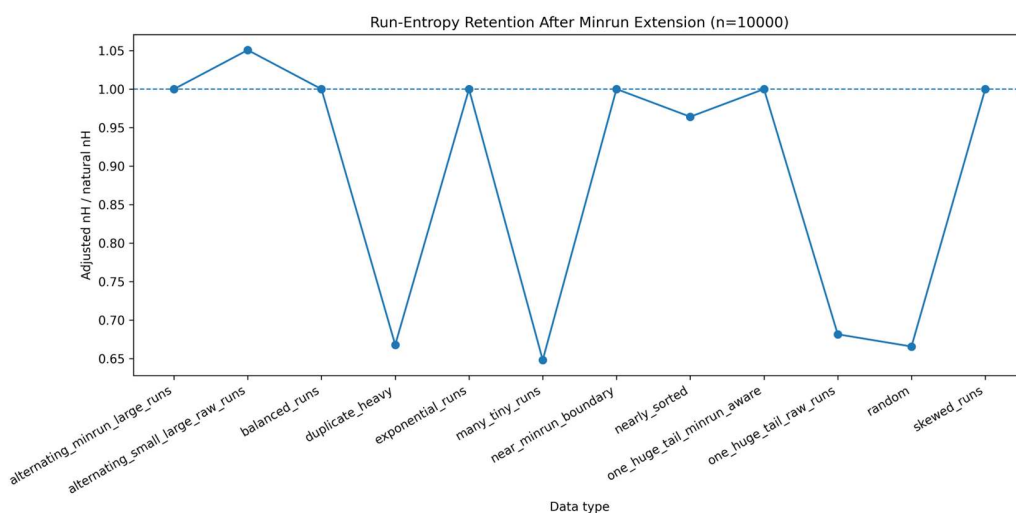


Figure A1. Run-entropy retention after minrun extension at $n = 10000$.

random、duplicate_heavy 與 many_tiny_runs 的 entropy retention 約為 0.65 至 0.67；alternating_small_large_raw_runs 則略高於 1，表示 minrun preprocessing 將原本極端交替的 run-length distribution 轉換成更接近 equal-length、entropy 更高的 adjusted distribution。這也說明 entropy change 不應一律被解讀為 information loss。

A.2 Natural and Adjusted Run Counts

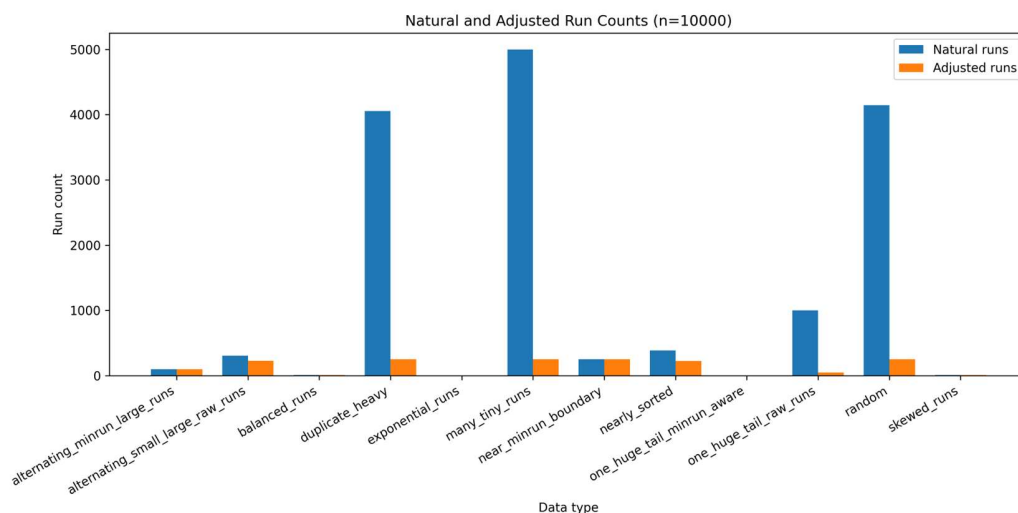


Figure A2. Natural and adjusted run counts after minrun preprocessing at $n = 10000$.

Figure A2 顯示 minrun 對 highly fragmented inputs 的 compression effect。相較之下，balanced、exponential、minrun-aware alternating 與 skewed cases 的 natural 與 adjusted run counts 相同，表示 minrun extension 沒有介入。

Appendix B. Scatter-Plot Analyses

B.1 Adjusted Run CV

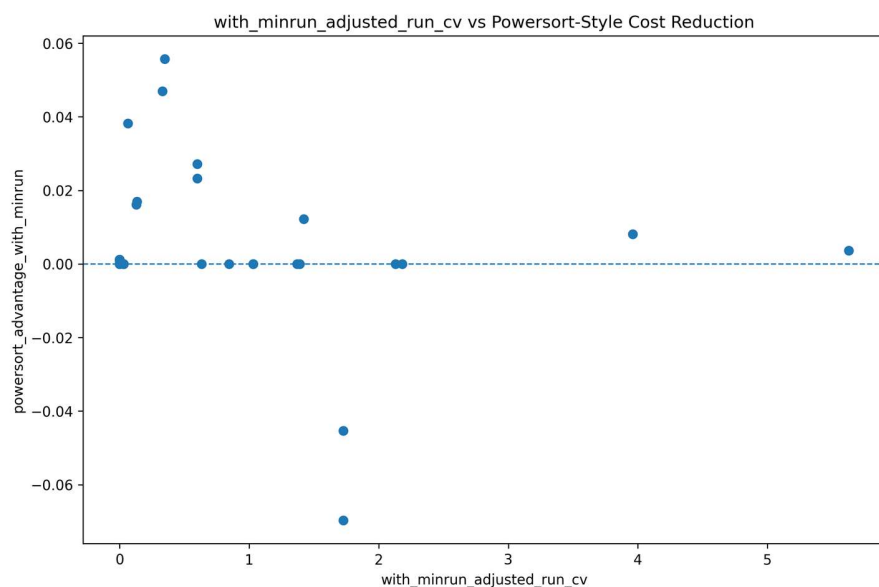


Figure B1. Adjusted run coefficient of variation versus with-minrun cost reduction.

High run CV 並不保證 positive cost reduction。兩個最明顯的 negative values 位於 medium-to-high CV region，而 extremely high CV 的 one-huge-tail cases 接近 0，支持 dominance 與 useful heterogeneity 不相同的 interpretation。

B.2 Adjusted Max-Run Ratio

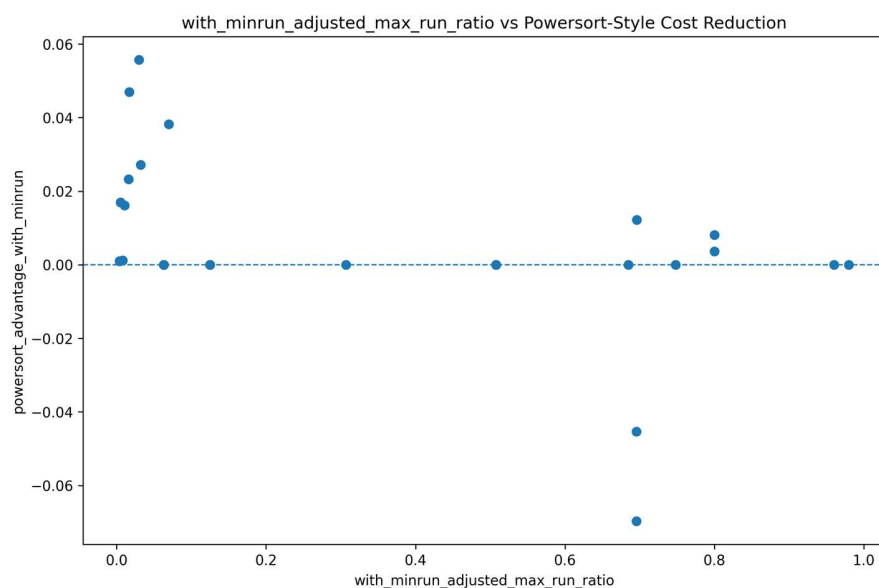


Figure B2. Adjusted maximum-run ratio versus with-minrun cost reduction.

整體呈 negative association，但 scatter 仍相當大。Max-run ratio 可描述 dominance，卻無法表達其餘 runs 的 positions 與 local length ratios，因此不宜單獨用來預測 policy gap。

Appendix C. Reproducibility

本報告所使用的主要 files 包含：

- run_distribution_analysis_cleaned.py
- powersort_advantage_profile.csv
- run_feature_correlation_with_advantage.csv
- assets/ 中的所有 figures

執行方式：

```
python run_distribution_analysis_cleaned.py
```